

Задача AB23. ТРИАГОЛНИК

🕒 10 сек. 📄 1024 MB

Новото раководство на националниот комитет, претставено од координаторите на групите A и B, сака да одржи строга програма на зададени теми при изборот на националниот тим. За таа цел, ќе треба да зададат задача по геометрија. Освен тоа – бројот на интерактивни задачи зададени досега е под очекуваното ниво. За да ја поправи кризата во изборот за IOI, Сашка одлучи да зададе задача што е и интерактивна и геометриска. Нејзиниот текст е следниот:

Жирито има сокриено пермутација $P_0, P_1, \dots, P_{N-1}$ на множеството $\{1, 2, 3, \dots, N\}$. Ваша задача е да ја пронајдете пермутацијата. За таа цел, може да му го поставувате на жирито следното прашање: “Дали е можно да се формира триаголник со позитивна плоштина со страни P_A, P_B, P_C ?”

Забележете дека е познато (според неравенствата за триаголник) дека тоа е можно ако и само ако:

$$P_A + P_B > P_C, P_A + P_C > P_B \text{ and } P_B + P_C > P_A.$$

Напишете програма **triangle**, која содржи функција **solve**, што ќе се компјилира заедно со програмата на жирито и ќе комуницира со неа преку поставување прашања од погоре опишаниот тип. На крајот од своето извршување, програмата мора да ја одреди пермутацијата.

Implementation details

Треба да ја имплементирате функцијата **solve**:

```
std::vector<int> solve(int N)
```

- N : должина на пермутацијата.

Оваа функција ќе биде повикана T пати по тест – еднаш за секој подтест, секојпат со исто N , и треба да ја врати сокриената пермутација за соодветниот подтест. За да го направите тоа, вашата програма може да ја повикува функцијата на жирито **query**:

```
bool query(int A, int B, int C)
```

- A, B, C : индекси на страните P_A, P_B, P_C .

Функцијата враќа **true** ако постои триаголник со позитивна плоштина со страни P_A, P_B, P_C , а **false** во спротивно.

Constraints

- $N = T = 1\,000$

Subtask

XVII МЕЃУНАРОДЕН НАПРЕДЕН ТУРНИР ПО ИНФОРМАТИКА
ОНЛАЈН 2026

Потзадача	Поени	Опис
1	100	Потзадачата се состои од 1 тест со $T = 1\,000$ пермутации, избрани случајно и рамномерно од множеството на сите можни пермутации, при што секоја е со $N = 1\,000$ елементи.

Поените за дадена потзадача се добиваат само ако успешно се поминати сите тестови за неа.

Scoring

Нека $Q_{contestant}$ е просечниот број прашања што вашата програма ги поставува при едно повикување на `solve` на единствениот тест, и нека $Q_{target} = 8770$. Тогаш вашиот резултат за задачата ќе биде:

$$\begin{cases} 0 & \text{if } Q_{contestant} > 2 \times 10^6, \\ 100 & \text{if } Q_{contestant} \leq Q_{target}, \\ 100 \times \max \left(0.15, 1 - \sqrt{1 - \left(\frac{Q_{target}}{Q_{contestant}} \right)^{0.55}} \right) & \text{if } Q_{contestant} > Q_{target}. \end{cases}$$

Sample interaction

За оваа примерна интеракција важи $N = 4$, $T = 2$.

Акција на натпреварувачот	Акција на жирито
	<code>solve(4)</code>
<code>query(0, 0, 0)</code>	<code>true</code>
<code>query(0, 1, 2)</code>	<code>false</code>
<code>query(0, 1, 3)</code>	<code>false</code>
<code>query(0, 2, 3)</code>	<code>true</code>
<code>query(1, 2, 3)</code>	<code>false</code>
<code>return {3, 1, 2, 4};</code>	
	<code>solve(4)</code>
<code>query(0, 1, 2)</code>	<code>false</code>
<code>query(0, 1, 3)</code>	<code>true</code>
<code>query(0, 2, 3)</code>	<code>false</code>
<code>query(1, 2, 3)</code>	<code>false</code>
<code>return {4, 2, 1, 3};</code>	

Local testing

Формат на влез:

- линија 1: три цели броеви T , N и R - бројот на подтестови, големината на пермутациите и режимот на извршување. Ако $R = 1$, локалниот проверувач ќе генерира рамномерно случајни пермутации и ќе очекува број на втората линија - S , кој ќе биде семе за неговиот случаен генератор. Ако $R = 2$, влезот продолжува на следниов начин:
- линиите 2 до $(1 + T)$: пермутации на $\{1, 2, \dots, N\}$.

Формат на излез:

- линија 1: порака за грешка или просечниот број прашања за подтестовите, ако сите пермутации се точно одредени.